The extracted code refines the Braid machine

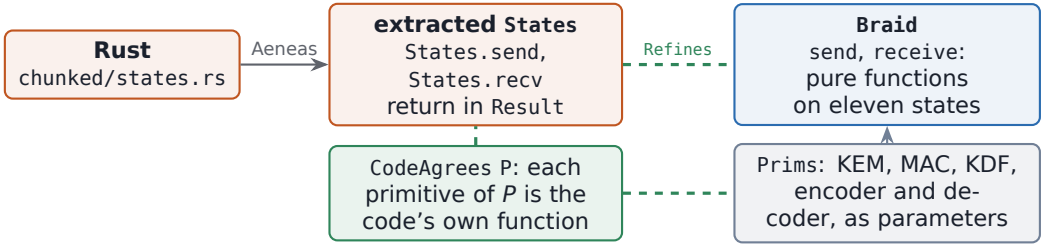
SpqrBridge slides, the refinement in spqr-verify

SpqrBridge

November 2026

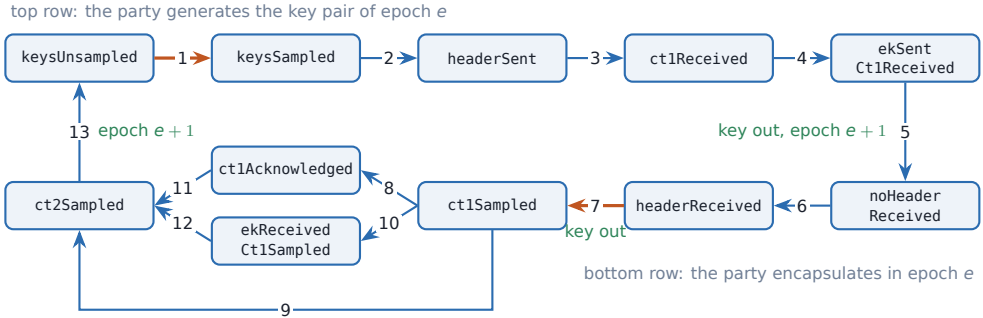
What does spqr-verify prove about the extracted code, and what does it leave open?

From Rust to an abstract machine



- `Braid` is the state machine of ML-KEM Braid §2.5, written by hand in Lean. It has no `Result`, no machine integers and no RNG state. `send` takes the random bytes as an argument.
- `Braid` does not fix the cryptography. It takes a record `Prims`. `CodeAgrees P` says that *P* computes what the extracted functions compute, and `codeAgrees_exists` proves that such a *P* exists.
- The refinement theorems hold for every *P* with `CodeAgrees P`. They say nothing about what the primitives compute. That is the task of the proofs about libcrux.
- The theorems are in spqr-verify, `Spqr/Specs/Braid/StateMachine`. The bridge imports them.

The machine: eleven states, thirteen transitions



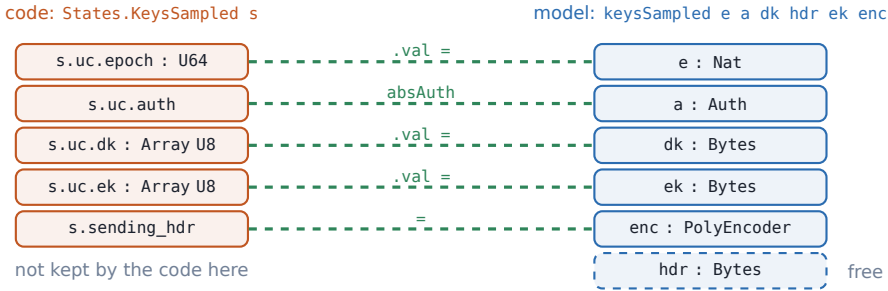
send, reads bytes
1 `KeyGen`
7 `Encaps1`

receive, top row
2 first `ct1` chunk
3 `ct1` complete 4 first `ct2` chunk
5 `ct2` complete, MAC correct

receive, bottom row
6 header complete, MAC correct
8 ack, `ek` incomplete 12 ack
9 ack, `ek` complete 10, 11 `ek` complete

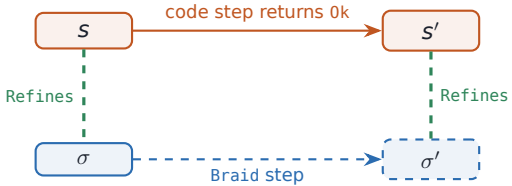
ack: `ct1` is acknowledged. 13: a message of epoch $e+1$. Not drawn: a step that keeps the state (every other `send` sends the next chunk, and a `receive` can complete nothing).

`Refines`: the same state, with equal values



- `Refines s σ` has one case for each of the eleven states. A pair of different states is `False`. The model uses the code's own encoder and decoder types.
- Five cases also fix sizes. Examples: the `ct2` decoder needs 80 points, and a received `ct1` has 960 bytes.
- The sizes are necessary. A `ct2` decoder that needs 65 points completes with a 130-byte message, and the code can accept its 2-byte MAC. The model refuses it.
- So `Refines` holds on states that the code builds from `init`. A state restored from bytes has no such proof.

The refinement theorems



Given: the solid parts.
Proved: the dashed parts exist, with the same message (`absMsg`) and the same key (`absKey`).

`init_refines`	`init_a` and `init_b` refine `initAlice` and `initBob`.
`send_refines`	If `States.send` returns `Ok`, then `Braid.send σ r` matches it for some bytes r.
`recv_refines`	If `States.recv` returns `Ok`, then `Braid.receive` accepts the message and matches it. Needs `MlkemSizes`.
`run_refines`	A run of the code from s gives a run of `Braid` from σ with the same keys, by induction on the run.

- The theorems go in one direction: from a successful code step to a model step. They do not say that the code succeeds, and they do not name r.

Three named deviations from §2.5

The code differs from ML-KEM Braid §2.5 in three places. `Braid` follows the code at each one, so that the refinement holds. Each is a constructor of `Braid.Deviation`.

	§2.5	code, and `Braid`
`futureEpochError`	A later-epoch message is ignored, except at transition 13.	`recv` returns `Err(EpochOutOfRange)`, and the model returns `none`.
`ct1AckAccepted`	`EkSentCt1Received` sends a message of type `None`. `Ct1Ack` is not used.	It sends `Ct1Ack(true)`, and `EkReceivedCt1Sampled` takes transition 12 on it.
`lateEkChunk`	`Ct1Acknowledged` drops a plain `Ek` chunk.	The chunk goes to the `ek` decoder, and can complete transition 11.

- A declaration that follows the code at a deviation carries `@[braid_deviation …]`. A search for the tag gives every place.
- The docstring of each constructor gives the effect in a normal run. That reasoning is not proved.
- SM's `SPQR.Chunked` has the same deviations. `lateEkChunk` is the reason that `SPQR.correctness_error_le` relabels one delivery when it translates to Braid.

What the refinement gives, and what is open

proved in spqr-verify

`init_refines`
`send_refines`: for some bytes r
`recv_refines`: with `MlkemSizes`
`run_refines`

Direction: the code succeeds $\Rightarrow$ the model steps.

open, premises of the bridge

`mlkem_sizes` (#786): three libcrux sizes
`send_total` (#785): a reachable send succeeds
`recv_complete` (#785): the model accepts $\Rightarrow$
the code succeeds, with the same result
`send_refines_bytes` (#787): r is the RNG's answer

- The bridge composes `Refines` with its relation from `Braid` to `SPQR.Chunked`: $RA\ s\ t \iff \exists \sigma,$ `Refines` $s\ \sigma \wedge$ `ChunkedRel` $\sigma\ t$ (main deck, "The chain of relations").
- The probability bound needs both directions. If the model accepts a message and the code refuses it, the code's game fails where SM's game continues. `recv_complete` and `send_total` close that direction on reachable states.
- `send_refines_bytes` gives the distribution of r (probability deck).
- The four open facts are the fields of `BridgeObligations`.